



Bootstrapping Coding Agents: The Specification Is the Program

Martin Monperrus¹, KTH Royal Institute of Technology, Stockholm, Sweden

A **CODING AGENT** can bootstrap itself. Starting from a 926-word specification and a first implementation produced by an existing agent (Claude Code), a newly generated coding agent re-implements the same specification correctly from scratch. This reproduces, in the domain of AI coding agents, the classical bootstrap sequence known from compiler construction, and instantiates the meta-circular property known from Lisp. The result carries a practical implication: The specification, not the implementation, is the stable artifact of record. Improving an agent means improving its specification; the implementation is, in principle, regenerable at any time.

Introduction

Coding agents are programs that accept natural-language task descriptions and produce or modify source code. Teams now deploy them to write tests, perform refactoring, and implement features from natural language task descriptions and requirements.^{1,2}

Compiler writers discovered decades ago that a new language

implementation passes a meaningful milestone when it can compile itself. This property, called *self-hosting*, is not merely a curiosity: It validates that the implementation is expressive enough to describe its own behavior. A self-hosting compiler is also a fixed point of the compilation process: It reproduces itself under its own translation.

The same milestone has now been reached for AI coding agents. Given only a natural-language specification, a coding agent can implement itself. This article describes the experiment, draws the analogy to classical results in computer science, and examines the implications for software engineering practice.

The Bootstrapping Experiment

The experiment consists of three steps, each building on the previous one. All artifacts are publicly available in the **meta-circular** repository.³

The coding-agent bootstrap sequence. Claude Code implements $agent_0$ from the specification; $agent_0$ then bootstraps $agent_1$ from the same specification. Dashed arrows show where the specification is consumed. Both implementations satisfy s , denoted $\models s$.

Step 1: Specification

A large language model (LLM) was prompted to write a specification for a coding agent: a program that receives a task description in natural language and produces or modifies source code to perform the task. The resulting document defines the agent's interface, its expected behavior, and the constraints it must respect. The specification is 926 words long and is available at the repository listed in the references.³ It covers three areas: the agent's *interface* (command-line arguments, environment variables, and application programming interface (API) interaction), its *behavioral constraints* (how it handles multiturn conversations, tool use, and error conditions), and the *tool loop* (the cycle of receiving a task, calling tools, observing results, and producing output). The document is intentionally compact: At 926 words, it can be read and audited by a single engineer in under five minutes.

Step 2: First Implementation

Claude Code with Sonnet 4.6,⁴ an existing coding agent, was given the specification and asked to implement it. The prompt was minimal:

implement the spec in a single python file

Digital Object Identifier 10.1109/MS.2026.3687051
Date of current version: 30 June 2026

The result is a working Python program (agent.py) that satisfies the specification. No manual edits were required. The program accepts a task description as a command-line argument and uses a language model API to produce or modify source code:

```
$ python agent.py
usage: agent.py [-h] [--model MODEL]
               [--base-url BASE_URL] [--api-key API_KEY]
               [--max-turns MAX_TURNS] [--cwd CWD]
               [task]
```

Step 3: Self-Implementation

The newly generated agent was given the same specification and asked to implement it again, using the same prompt as Step 2. It succeeded. The agent implemented itself:

```
$ python agent.py \
  "implement the spec in a single python file"
```

The output is a new agent.py that satisfies the same specification. Both the first-generation and second-generation programs were verified manually against the specification (Figure 1).

Meta-Circularity

The term *meta-circular* comes from the Lisp tradition. In 1960, McCarthy presented a Lisp interpreter written in Lisp itself.⁵ The interpreter is called meta-circular because the language in which it is written is the same language it interprets. The implementation and the object of interpretation are drawn from the same formalism.

The first practical self-hosting Lisp compiler was implemented by Hart and Levin at MIT in 1962,⁶ two years after McCarthy's theoretical definition. Their work established the bootstrap sequence that compiler writers would follow for decades: Define the language formally, implement it in another host, then use that implementation to compile itself. More recently, Courant et al. reproduced the bootstrap for OCaml using the Camlboot project, demonstrating that a minimal, auditable compiler can find its full industrial counterpart.⁷ This creates a clear lineage between the meta-circularity definition,⁵ to the coding-agent bootstrap (2026).

The coding-agent experiment follows the same pattern. The agent generated in Step 2 is, in this terminology,

a meta-circular agent: It is an implementation of "coding-agent behavior" produced by a system that itself exhibits "coding-agent behavior." When the Step-2 agent reproduces itself in Step 3, it demonstrates meta-circularity in the operational sense. The agent is a fixed point of its own implementation procedure.

To recap, let F be the function that maps a specification and a coding agent to an implementation satisfying that specification. Let s be the coding-agent specification. Then

$$\begin{aligned} \text{agent}_0 &= F(s, \text{ClaudeCode}) \\ \text{agent}_1 &= F(s, \text{agent}_0) \end{aligned}$$

This is illustrated in Figure 1. This is the coding-agent analog of a self-hosting compiler: The first generation is sufficient for the second generation to be indistinguishable from it under the specification. The specification, not the bootstrap tool, is the stable artifact.

Structural analogy between the classical compiler bootstrap ([Fig. 1(a)]) and the coding-agent bootstrap ([Fig. 1(b)]). The first self-generated artifact is equivalent to the first-generation one.

Implications

Specification as the Focus of Correctness

If a specification is sufficient to generate an agent, and the agent is sufficient to regenerate itself from the same specification, then the specification is the true focus of correctness. Improving the agent means improving the specification. The implementation is, in principle, disposable and regenerable at any time.

This reframes code review. Rather than auditing every line of an auto-generated implementation, reviewers should audit the specification from which the implementation is derived.⁸ Errors in the specification propagate

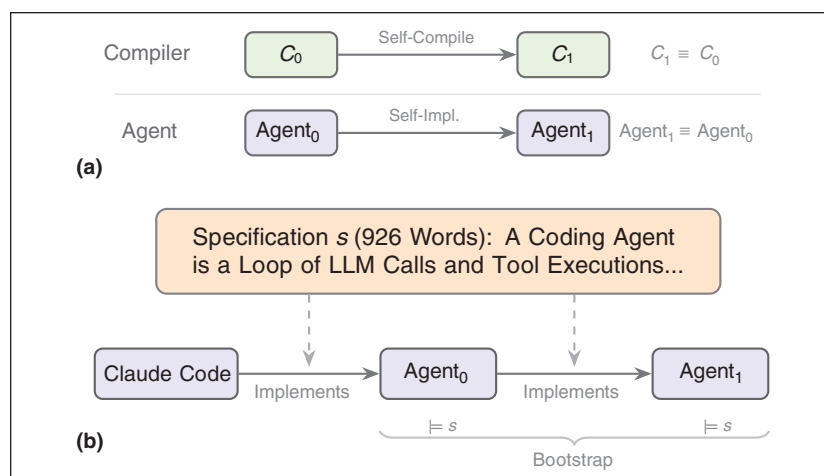


FIGURE 1. Analogy with compiler bootstrap (a) Agent₀ self-implements Agent₁ (b) such that they both satisfy specification S .

to every generation of the implementation or to every generation in different programming languages.

An industrial-scale instance of this pattern is described by Lopotolo et al.¹: A team of three to seven engineers built a million-line codebase using Codex over five months, with zero manually written lines of code. The team treats their structured docs/directory as the reference system, not the code itself. The codebase is regenerable; the specification is the stable artifact. This mirrors, at industrial scale, the implication derived from the bootstrap experiment.

This result aligns with the emerging practice of spec-driven development,⁹ which treats the specification as the primary engineering artifact and generated code as a build product. The practical consequences are concrete. In version control, the specification belongs on the main branch and is subject to the full review workflow; generated implementation files may be treated as build artifacts, reconstructed on demand rather than tracked commit-by-commit. In team workflows, specification review replaces code review as the primary quality gate. An engineer who can evaluate whether a specification correctly and completely describes the intended behavior provides more durable value than one who audits lines of generated code that may be regenerated wholesale by the next model update.

Good Specifications for AI Coding

The bootstrap result raises a practical question: What properties must a specification have to serve as the stable artifact of record?

Size and Auditability

The 926-word specification used here is small enough to read in a single sitting.

A useful budget: Keep the specification under 1,500 words or under 15 min of reading time. If a section requires cross-referencing another section to be understood, split or restructure.

Behavioral Completeness

The specification must describe every capability the agent exercises. A practical checklist: For each tool call, API interaction, and error condition in the generated code, ask “is this behavior described in the specification?” Any “no” is a gap. Write the specification first and treat undescribed behavior as intentionally undefined.

Convergence Across Implementations

A good specification is unambiguous enough that two independent implementations converge on the same observable behavior. Heuristic: Give the specification to two different engineers (or two independent LLMs¹⁰) and check whether the results agree on externally visible behavior such as command-line interface and output format. This is the N-version programming principle applied to specification validation¹¹: divergence between independently produced implementations reveals ambiguity that must be resolved in the specification.

Abstraction Level

A specification should describe *what* the agent does, not *how*. If a sentence names a data structure, a library, or a language feature, it is probably too low-level. Sentences that name only inputs, outputs, and conditions are at the right level and will remain stable across model generations.

Trusting Trust and AI Coding

Thompson’s 1984 Turing Award lecture showed that a trojan horse in a compiler binary can reproduce itself through every subsequent

compilation without appearing in any source file.¹² The coding-agent bootstrap introduces an analogous structure. A compromise in the model weights, the API endpoint, or the execution environment can propagate silently through every generated implementation. Like a compiler binary, model weights are opaque and are updated out-of-band by a third party.

The practical response is to treat each layer as a versioned, auditable dependency. Pin the model version used for generation, run generation in a controlled CI environment whose executions are fully logged, record the full trajectories. The specification remains the only artifact that a human engineer can read and audit directly; everything downstream of it is a build product that should be reproducible from a fixed, declared set of inputs.

Limitations

The bootstrap experiment demonstrates a property; it does not resolve all questions about the technique’s generality.

Complexity Scaling

The 926-word specification is simple. Whether the technique scales to specifications of 10,000 or 100,000 words is an open question. The Attractor case (34,900 words) provides evidence that longer specifications remain tractable, but verification difficulty grows: The test suite must cover a larger behavioral surface, and the specification itself may harbor internal inconsistencies that only manifest in complex interactions.

Model Dependency

The bootstrap succeeds only with recent state-of-the-art models. Earlier or smaller models generate either syntactically invalid or behaviorally incorrect implementations. The



ABOUT THE AUTHOR



MARTIN MONPERRUS is a professor of software technology at KTH Royal Institute of Technology, 100 44 Stockholm, Sweden. He is a Fellow of IEEE. Contact him at monperrus@kth.se.

bootstrapping technique requires a model capable of faithfully executing a natural-language specification; this capability is not guaranteed to be stable or universal across model families and has only been reached recently.

Related Work

The Attractor project by StrongDM¹³ applies the same idea principle at a larger scale. Attractor's specification is approximately 34,900 words; the specification used in the present experiment is 926 words, roughly 38 times shorter. The smaller scale makes the present repository more suitable for learning and for reproducing the bootstrap property from scratch. A notable feature of the Attractor repository is that it publishes only the specification, treating the implementation as a regenerable artifact. Our accompanying open-science repository also includes the generated agent code to serve both as scientific evidence and learning material.

A coding agent can bootstrap itself. Starting from a natural-language specification and a first implementation produced by an existing agent, a newly generated agent reimplements the same specification correctly. The first self-generated implementation satisfies the same specification as the first-generation implementation, verified by manual inspection.

This result reproduces, in the domain of AI coding agents, the classical bootstrap sequence from compiler construction, and instantiates the meta-circular property from Lisp. The practical implication is a shift in the focus of engineering effort: The specification is the artifact of record, and the implementation is regenerable on demand. 

Acknowledgment

The author thanks StrongDM for the Attractor project, the participants of the Bellairs 2026 Workshop on Continuous Software Engineering, and the Anthropic AI coding team. Generative AI has been used to edit this article.

References

1. R. Lopopolo, "Harness engineering: Leveraging Codex in an agent-first world," *OpenAI Engineering Blog*, Feb. 11, 2026. [Online]. Available: <https://openai.com/index/harness-engineering/>
2. C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.
3. M. Monperrus, "The meta-circular coding agent." GitHub. Accessed: Apr. 1, 2026. [Online]. Available: <https://github.com/ASSERT-KTH/meta-circular>
4. N. Courant, J. Lepiller, and G. Scherer, "Debootstrapping without archeology: Stacked Implementations in Camlboot," *Art. Sci., Eng. Program.*, vol. 6, no. 3, 2022, Art. no. 13, doi: [10.22152/programming-journal.org/2022/6/13](https://doi.org/10.22152/programming-journal.org/2022/6/13).
5. J. McCarthy, "Recursive functions of symbolic expressions and their computation by machine, Part I," *Commun. ACM*, vol. 3, no. 4, pp. 184–195, 1960, doi: [10.1145/367177.367199](https://doi.org/10.1145/367177.367199).
6. T. P. Hart and M. Levin, *The New Compiler*. AIM-39. Cambridge, MA, USA: Massachusetts Inst. of Technol., 1962.
7. D. Kesner, "A fine-grained computational interpretation of Girard's intuitionistic proof-nets," *Proc. ACM Programming Lang.*, vol. 6, no. POPL, pp. 1–28, 2022, doi: [10.1145/3498669](https://doi.org/10.1145/3498669).
8. A. Osmani, "Code review in the age of AI." Elevate · Substack. Accessed: Apr. 1, 2026. [Online]. Available: <https://addyo.substack.com/p/code-review-in-the-age-of-ai>
9. R. Piskala, "Spec-driven development: From code to contract in the age of AI coding assistants," 2026, *arXiv:2602.00180*.
10. J. Ron, D. Gaspar, J. Cabrera-Arteaga, B. Baudry, and M. Monperrus, "Galápagos: automated N-version programming with LLMs," *ACM Trans. Software Eng. Method.*, early access, Dec. 17, 2025, doi: [10.1145/3785363](https://doi.org/10.1145/3785363).
11. A. Avizienis, "The N-version approach to fault-tolerant software," *IEEE Trans. Softw. Eng.*, vol. SE-11, no. 12, pp. 1491–501, Dec. 1985, doi: [10.1109/TSE.1985.231893](https://doi.org/10.1109/TSE.1985.231893).
12. K. Thompson, "Reflections on trusting trust," *Commun. ACM*, vol. 27, no. 8, pp. 761–763, 1984, doi: [10.1145/358198.358210.s](https://doi.org/10.1145/358198.358210.s).
13. "StrongDM/Attractor." GitHub. Accessed: Apr. 1, 2026. [Online]. Available: <https://github.com/strongdm/attractor>